

# Apache HTTP Server

Study Summary – Chapters 1–8

**FWD 213 – Web Administration**

Ch1: Introduction • Ch2: Installation • Ch3: Configuration Files  
Ch4: Virtual Hosts • Ch5: SSL • Ch6: Forward Proxy  
Ch7: Reverse Proxy • Ch8: Server Statistics

Prepared: June 2026

## Chapter 1: Introduction to Apache HTTP Server

---

### Key Definitions

**Apache HTTP Server:** An open-source web server for Unix/Linux/Windows; derived from 'Patchy', meaning incomplete — built from **patches** applied to NCSA httpd.

**HTTP (Hypertext Transfer Protocol):** An application-layer protocol built on top of TCP enabling browser-server communication via GET, POST, and HEAD request/response messages.

**SSL (Secure Sockets Layer):** A **security** protocol that **encrypts** client/server communication; runs on top of TCP/IP using public **keys**, symmetric **keys**, and **certificates**.

**ARP (Address Resolution Protocol):** A low-level Layer-2 protocol that converts an IP address into its corresponding physical (MAC) network address.

**DHCP (Dynamic Host Configuration Protocol):** A protocol that automatically assigns a unique IP address to **any** device joining a network, then releases it when the device leaves.

**FTP (File Transfer Protocol):** A simple IP-based protocol that allows file transfer between two nodes on a network.

**MPM (Multi-Processing Module):** Platform-specific Apache modules that define how multiple simultaneous connections are handled on different operating systems.

### Key Facts

- HTTP uses TCP port 80 by default; HTTPS (SSL) **always** uses port 443.
- Apache powers over 67% of **all** web servers worldwide.
- HTTP message types: GET, POST, and HEAD.
- Apache 2.0 added hybrid multiprocess/multithreaded mode on Unix for improved scalability.

### Exam Tips & Traps

**Exam Tip:** HTTP = Application Layer (OSI Layer 7); ARP = Data Link Layer (OSI Layer 2). Know these layers.

**Exam Tip:** Apache 2.0 introduced PCRE (Perl-Compatible Regular Expressions) for more powerful pattern matching.

**Exam Trap:** HTTP does **NOT** provide **security** — SSL/TLS **must** be added separately. **Never** confuse HTTP with HTTPS.

**Exam Trap:** ARP resolves IP → MAC (**not** MAC → IP). RARP does the reverse. Do **not** confuse them.

## Chapter 2: Installing Apache on Linux and Windows

---

### Key Definitions

**httpd:** The Apache HTTP Server daemon; version 2.4.56 (released 2023-03-07) is the latest stable release.

**Static Website:** A site built with **only** HTML/CSS that **always** delivers the same content to **every** visitor.

**Dynamic Website:** A site that generates content at runtime using a database and a server-side language (PHP, Python, NodeJS).

**XAMPP:** A pre-built installer bundling Apache, MariaDB, PHP, and Perl for Windows, macOS, and Linux; **not** for production.

**Source Code Installation:** Compiling Apache from official source using 'make' then 'make install'; maximum customization but complex.

**Package Manager Installation:** Installing Apache via apt-get (Ubuntu/Debian) or yum/dnf (Fedora/RHEL) — quickest and easiest method.

### Key Commands

```
sudo apt-get install apache2          # Ubuntu/Debian
sudo systemctl start|stop|restart apache2
sudo systemctl enable apache2         # Enable at boot
sudo systemctl status apache2         # Check status
docker run -d --name myserver -p 80:80 httpd # Docker
```

### Exam Tips & Traps

**Exam Tip:** On Ubuntu/Debian the service is 'apache2'; on RHEL/CentOS it is 'httpd'. Commands differ between distros.

**Exam Tip:** Apache **must** be restarted for main configuration file changes; **only** .htaccess changes apply immediately.

**Exam Trap:** Source code installation does **NOT** use apt-get or yum — **never** confuse it with package manager installation.

**Exam Trap:** XAMPP is **only** for development/testing — **never** deploy it in a production environment.

## Chapter 3: Configuration Files

---

### Key Definitions

**httpd.conf / apache2.conf:** The main Apache configuration file; changes are **only** recognized when the server is started or restarted.

**Directive:** A configuration command (name + space-separated arguments) placed one per line in a plain text config file.

**DSO – Dynamic Shared Object:** An external module dynamically loaded into httpd at runtime using the LoadModule directive.

**.htaccess File:** A per-directory config file for decentralized management; changes take immediate effect as it is read on **every** request.

**AllowOverride:** Directive that controls which directives are permitted in .htaccess files; set to None to completely ignore .htaccess files.

**Container:** Applies directives to URL paths (webpace); should **never** be used to **restrict** access to filesystem objects.

**Container:** Applies directives to filesystem directories and **all** subdirectories; the correct way to **restrict** filesystem access.

### Container Summary

- / — filesystem directories (and regex variant).
- / — files by name or regex pattern.
- / — URL paths in webspace.
- // — conditional; evaluated **only** at startup.

### Key Commands

```
apachectl configtest          # Test config syntax without restarting
apachectl -l                  # List statically compiled modules
apachectl -M                  # List all modules (static + dynamic)
LoadModule status_module modules/mod_status.so
Define servername www.example.com
```

### Exam Tips & Traps

**Exam Tip:** Directives are case-insensitive, but their arguments are often case-sensitive.

**Exam Tip:** Use 'apachectl configtest' **before** restarting to catch syntax errors without downtime.

**Exam Trap:** **Never** use to **restrict** access to filesystem objects — different URL paths can map to the same file, **bypassing** the **restriction**.

**Exam Trap:** Comments **cannot** share a line with a directive. A '#' on the same line as a directive does **NOT** comment it out.

## Chapter 4: Virtual Hosts

---

### Key Definitions

**Virtual Host:** Configuration that allows multiple websites to run on a single physical server, each with its own directory.

**Name-based Virtual Hosting:** Multiple websites share the same IP address; the server uses the HTTP Host header to identify the site.

**IP-based Virtual Hosting:** Each website has a dedicated IP address; the server selects the virtual host based on the connection IP.

**ServerName:** Directive specifying the hostname (and port) used to identify the server within a VirtualHost **block**.

**ServerAlias:** Defines additional hostnames resolved to the same virtual host (e.g., www.example.com and example.com).

**DocumentRoot:** The filesystem directory from which Apache serves files for a virtual host; **must** be unique per site.

### Example VirtualHost Configuration

```
ServerName www.example.com

ServerAlias example.com

DocumentRoot /var/www/example

ErrorLog /var/log/apache2/example_error.log
```

### Exam Tips & Traps

**Exam Tip:** Name-based virtual hosting is more efficient — one IP can host many sites. **Always** prefer it over IP-based.

**Exam Tip:** The **first** VirtualHost **block** in the config file acts as the default for unmatched requests on that IP.

**Exam Trap:** IP-based hosting requires a separate IP per site — do **not** confuse it with name-based which shares one IP.

**Exam Trap:** Forgetting ServerAlias means requests to 'example.com' (without www) will **not** match the virtual host.

## Chapter 5: SSL Configuration

---

### Key Definitions

**SSL/TLS:** Protocol layer between TCP/IP and HTTP that provides mutual **authentication**, digital-signature **integrity**, and **encryption** for privacy; uses port 443.

**Symmetric (Conventional) Cryptography:** Both sender and receiver share the same secret **key** for **encryption** and **decryption**; fast but requires **secure key** exchange.

**Asymmetric (Public Key) Cryptography:** Uses two **keys**: Public **Key** (shared with everyone) to **encrypt**; Private **Key** (kept secret) to **decrypt**.

**Message Digest (Hash Function):** A one-way function producing a fixed-length summary; **any** change in the message changes the digest, ensuring **integrity**.

**Digital Signature:** Created by **encrypting** a message digest with the sender's private **key**; proves authenticity and **integrity**.

**Certificate Authority (CA):** A trusted third-party that **verifies** identities and signs digital **certificates** with its private **key**.

**SSL Handshake:** The sequence where client and server exchange **certificates**, negotiate cipher suites, and establish a session **key**.

### Key Commands

```
a2enmod ssl                # Enable mod_ssl (Ubuntu)
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365
SSLEngine on
SSLCertificateFile    /etc/ssl/certs/server.crt
SSLCertificateKeyFile /etc/ssl/private/server.key
```

### Exam Tips & Traps

**Exam Tip:** SSL **always** uses port 443; HTTP **always** uses port 80. Know both port numbers.

**Exam Tip:** Public **Key encrypts** data; Private **Key decrypts** it. Once a session is established it can be reused via a Session ID.

**Exam Trap:** **Never expose** or share the Private **Key** — **only** the Public **Key** is distributed. **Exposing** it **compromises all encrypted** communications.

**Exam Trap:** A self-signed **certificate** provides **encryption** but **NOT** CA-verified identity — browsers will **always** show a **security warning**.

## Chapter 6: Configure Apache as a Forward Proxy Server

---

### Key Definitions

**Proxy Server:** A server that routes traffic between clients and external systems; it can mask client IPs and **enforce security**.

**Forward Proxy:** Sits between the client and origin server; the client **must** be specially configured to send requests through it.

**ProxyRequests On:** Directive that activates Apache as a forward proxy; **must never** be enabled without **securing** the server to **prevent** open-proxy **abuse**.

**mod\_proxy:** The Apache module **required** for both forward and reverse proxy configurations.

**Residential Proxy:** A forward proxy with a real ISP-assigned IP address associated with a physical location.

**Datacenter Proxy:** A forward proxy whose IP addresses come from data centers (**not** ISPs); faster but easier to **detect**.

### Key Configuration

```
LoadModule proxy_module modules/mod_proxy.so
```

```
LoadModule proxy_http_module modules/mod_proxy_http.so
```

```
LoadModule proxy_connect_module modules/mod_proxy_connect.so
```

```
ProxyRequests On
```

```
ProxyVia On
```

```
Require ip 192.168.1.0/24 # only allow local subnet
```

### Exam Tips & Traps

**Exam Tip:** For HTTPS forward proxying, mod\_proxy\_connect **must** also be enabled in addition to mod\_proxy and mod\_proxy\_http.

**Exam Tip:** ProxyRequests On = forward proxy. In a reverse proxy setup this **must always** be set to Off.

**Exam Trap:** **Never** enable ProxyRequests On without access restrictions — it creates an open proxy that anyone can **abuse**.

**Exam Trap:** Forward proxy hides the CLIENT's identity. Reverse proxy hides the SERVER's identity. Do **not** confuse the two.

## Chapter 7: Configure Apache as a Reverse Proxy Server

---

### Key Definitions

**Reverse Proxy (Gateway):** Sits in front of backend servers; appears to clients as a normal web server while forwarding requests internally.

**ProxyPass:** Directive mapping a local URL path to a remote backend server URL, activating the reverse proxy.

**ProxyPassReverse:** Adjusts HTTP redirect headers from the backend so they point to the proxy URL, **not** the internal server.

**Load Balancing:** Distributing incoming requests across multiple backend servers to improve performance and **prevent** overloading.

**mod\_proxy\_balancer:** Apache module that enables load balancing across a cluster of backend servers using configurable algorithms.

### Key Configuration

```
ProxyRequests Off    # must be Off for reverse proxy

ProxyPass            /app http://backend:8080/app

ProxyPassReverse /app http://backend:8080/app

# Load balancer:

    BalancerMember http://server1:8080
    BalancerMember http://server2:8080

ProxyPass / balancer://mycluster/
```

### Reverse Proxy Use Cases

- Provide internet access to servers behind a firewall.
- Load balancing across multiple backend servers.
- SSL termination — handle SSL at the proxy, plain HTTP to backends.
- **DDoS protection** by shielding origin servers.

### Exam Tips & Traps

**Exam Tip:** **Always** use ProxyPassReverse alongside ProxyPass — without it, redirects from the backend **expose** internal addresses.

**Exam Tip:** Reverse proxy hides the SERVER; forward proxy hides the CLIENT.

**Exam Trap:** ProxyRequests **must** be Off in a reverse proxy setup — setting it On accidentally creates an open forward proxy.

**Exam Trap:** Do **not** confuse the [P] RewriteRule flag with ProxyPass — both can create a reverse proxy but are used in different contexts.



## Chapter 8: Display Server Statistics

---

### Key Definitions

**mod\_status:** An Apache Base module providing a real-time HTML page with server activity and performance statistics.

**server-status Page:** URL endpoint (e.g., /server-status) showing live server metrics; access **must** be **restricted** to trusted IPs.

**ExtendedStatus On:** Directive enabling detailed per-request statistics including CPU usage, requests/second, and bytes/second.

**mod\_info (server-info):** Apache module providing a comprehensive overview of server configuration, loaded modules, and **all** directives.

**Worker / Idle Worker:** A worker is a process/thread handling a request; idle workers are waiting. The ratio indicates server load.

### Statistics Provided by mod\_status

- Total requests, bytes transferred, server uptime, and restart time.
- CPU usage and server load average.
- Requests per second and bytes per second.
- Total idle and busy workers with PID and client info.

### Key Configuration

```
LoadModule status_module modules/mod_status.so
```

```
SetHandler server-status  
Require ip 127.0.0.1 ::1
```

```
ExtendedStatus On
```

```
# Browser: http://localhost/server-status
```

```
# Auto-refresh: http://localhost/server-status?refresh=5
```

```
# CLI:      apache2ctl status
```

### Exam Tips & Traps

**Exam Tip:** mod\_status is a Base module — included with Apache by default but **must** be enabled with a **block**.

**Exam Tip:** Add '?refresh=N' to the URL for live auto-refreshing monitoring (e.g., ?refresh=5).

**Exam Trap:** **Never expose** /server-status to the public internet — it reveals sensitive server info. **Always restrict** with 'Require ip'.

**Exam Trap:** mod\_status ≠ mod\_info: mod\_status shows live performance data; mod\_info shows static configuration details.